Large Language Models Empowered Personalized Web Agents

CSI5389 Paper Presentation

Sabrina Cai Eric Sun Yan Zhang

EECS, University of Ottawa
{hcai062, ysun338, Yan.Zhang}@uOttawa.ca

March 10, 2026

uOttawa

Presentation roadmap

Stage A — Sabrina

- Introduction
- Related work
- Task formulation

Stage B — Eric

- Benchmark construction
- Evaluation
- Benchmark analysis

Stage C — Yan

- **PUMA** framework
- Experiments
- Conclusion & future work

1 Background & Problem Statement

2 Benchmark Construction & Analysis

3 PUMA, Experiments & Conclusion

Why do we need personalized web agents?

Presenter: Sabrina

- Modern web services are powerful but **hard to navigate efficiently**.
- Existing web agents usually optimize for **instruction following**, not for **user preference matching**.
- Many real requests are *underspecified*:
  - "Find me a backpack"
  - "Recommend a product for my kitchen"
  - "Write a review for this item"
- These tasks only make sense when the agent also knows the user's:
  - purchase history,
  - budget and brand preferences,
  - writing tone and review habits.

Paper thesis

Better web agents must combine:

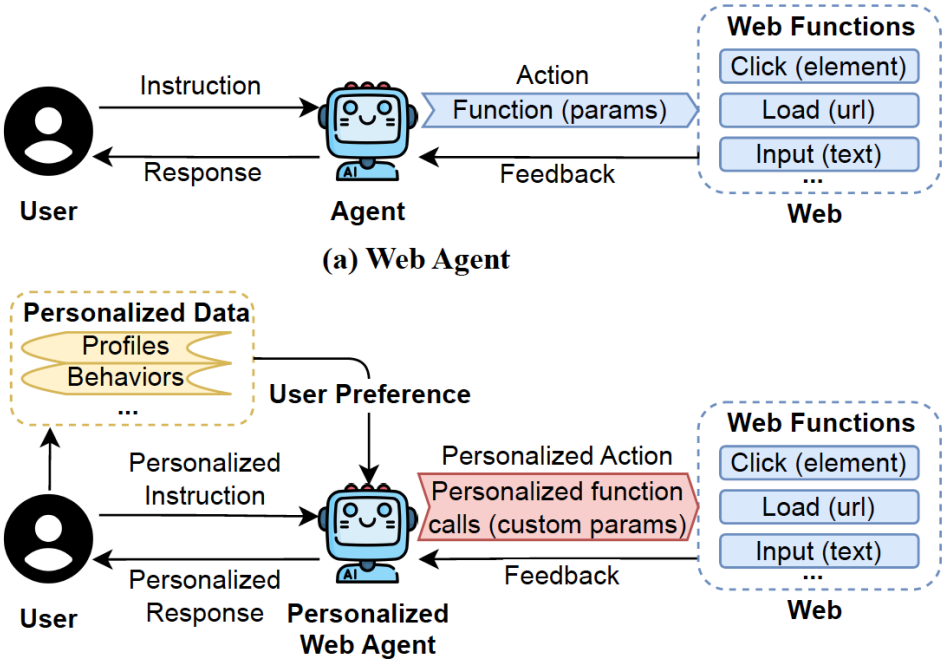
1. the current instruction,
2. personalized user data,
3. and the right web action.

Key shift

From **generic automation** to **user-aware automation**.

From traditional agents to personalized agents

Presenter: Sabrina



(a) Web Agent

(b) Personalized Web Agent

Traditional agent

- current query only
- general instruction following
- weak preference awareness

Personalized agent

- profile + behavior history
- hidden preference inference
- personalized action + parameters

Related work and the missing bridge

Presenter: Sabrina

Retyped from Table 1.

Benchmark	Interaction	Environment	Personalization
MiniWoB++	Single-turn	Mobile UI	No
WebShop	Single-turn	Shopping UI	No
WebArena	Single-turn	Web UI	No
WebLINX	Multi-turn	Web UI	No
ChatShop	Multi-turn	Web function	No
PersonalWAB	**Single + Multi**	**Web function**	**Yes**

- **Web-agent benchmarks** study browsing, navigation, or shopping.
- **Personalized LLM work** studies dialogue, content generation, and recommendation.
- The missing piece is **personalized function execution**: can an agent select the right web action and fill it with user-specific parameters?

Task formulation: what exactly is the agent solving?

Presenter: Sabrina

Personalized web interaction

Given a user instruction i, a user profile P_u, a behavior memory M_u, and callable web functions $\mathcal{F}$, the agent must output

$$f \in \mathcal{F}, \qquad \mathbf{p} = \text{personalized parameters},$$

so the returned result matches both the user's **explicit request** and **implicit preferences**.

Inputs

- User instruction
- Simulated user profile
- Historical web behaviors
- Available web functions

Outputs

- Correct function selection
- Correct parameter format
- Personalized result or response

Why this is harder than standard web agents

Two difficulties must be solved together: choosing a valid action and inferring preferences that are often **not explicitly stated**.

Main contributions of the paper

Presenter: Sabrina

1. **New task:** personalized web agents as a formal research problem.
2. **New benchmark: PersonalWAB**, covering personalized search, recommendation, and review generation.
3. **New method: PUMA** (Personalized User Memory-enhanced Alignment).
4. **Main result:** retrieving user history improves action selection and result quality, especially for **underspecified** instructions.

Take-home message

Personalization is both a **memory retrieval** problem and an **alignment** problem.

1 Background & Problem Statement

2 Benchmark Construction & Analysis

3 PUMA, Experiments & Conclusion

RL-style view: priors, environment, and algorithm

Presenter: Eric

Why this benchmark matters beyond this paper

In an RL-style view, agent capability comes from performance $\approx$ **priors** $+$ **environment** $+$ **algorithm**. This paper contributes to *all three*, and especially clarifies the **environment** for personalized web agents.

Priors

- pretrained language priors
- strong tool-use and reasoning ability
- examples: **LLaMA-2-7B** for PUMA; **gpt-4o / gpt-4o-mini** as strong LLM backbones

Environment

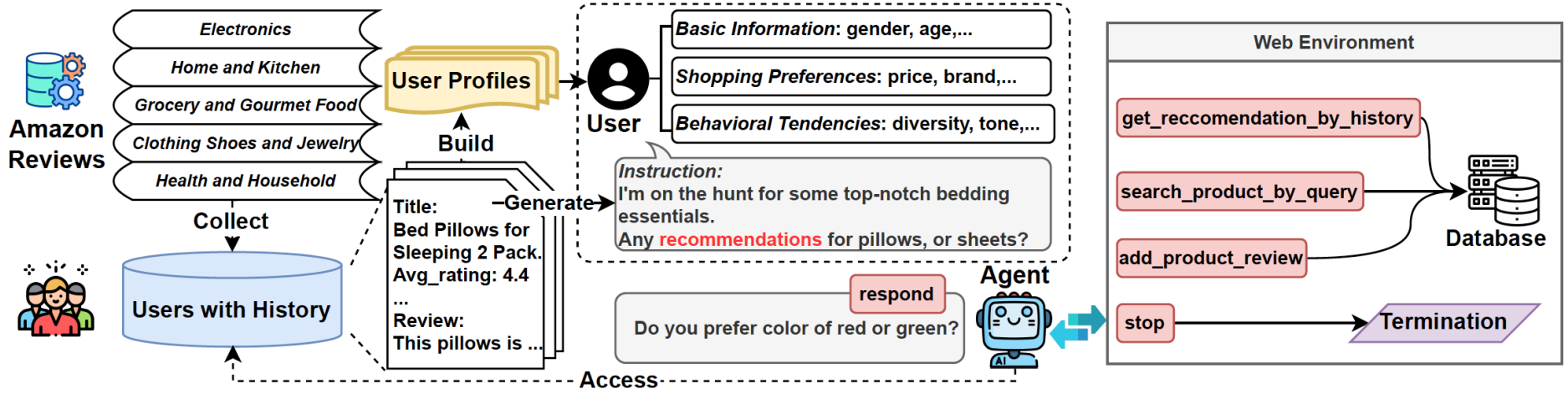
- **Benchmark: PersonalWAB**
- tasks: search, recommendation, review
- interface: `search_product_by_query`
- `get_recommendations_by_history`
- `add_product_review`, `respond`, `stop`

Algorithm

- proposed method: **PUMA**
- function identification
- task-specific memory retrieval
- parameter optimization with **SFT + DPO**

PersonalWAB: benchmark construction pipeline

Presenter: Eric



How to read the pipeline

- Built from **Amazon Review** data rather than expensive real-time user studies.
- Pipeline has three main stages: personalized data construction, instruction creation, and web environment implementation.
- The objective is to isolate the **personalization challenge** in a controlled setting.

How the benchmark data is built

Presenter: Eric

- **1,000 users** are sampled from Amazon Reviews.
- Five product categories:
  - Electronics
  - Home and Kitchen
  - Grocery and Gourmet Food
  - Clothing, Shoes, and Jewelry
  - Health and Household
- Chronological split per user:
  - 80% history
  - 10% train
  - 10% test

User profile dimensions

- basic info
- shopping preferences
- behavioral tendencies
- tone and style
- item references
- focus aspects in reviews

Three tasks and a function-based web environment

Presenter: Eric

Task families

- **Search**: produce a personalized query
- **Recommendation**: choose relevant products from history
- **Review**: generate a personalized review text

Implemented web functions

- `search_product_by_query`
- `get_recommendations_by_history`
- `add_product_review`
- `respond`
- `stop`

Why functions instead of full GUIs?

The authors abstract away noisy web interfaces so evaluation focuses on whether the agent chooses the **right personalized action**.

Key design choice

The benchmark tests **personalization ability**, not browser navigation skill.

Evaluation protocol

Presenter: Eric

Two tracks

- **Single-turn**: one chance to act
- **Multi-turn**: agent can ask/respond and receive simulator feedback

Metrics

- **Function accuracy**: right function + valid parameter format
- **Result accuracy**: quality of returned result
- **Average steps**: multi-turn efficiency

Search / recommendation result accuracy

$$\mathrm{ResAcc} = \begin{cases} 1 - \dfrac{r-1}{10}, & r \leq 10, \\ 0, & r > 10. \end{cases}$$

- r is the rank of the target product in the returned list.
- For review generation, the paper instead uses sentence-level similarity.

What does PersonalWAB contain?

Presenter: Eric

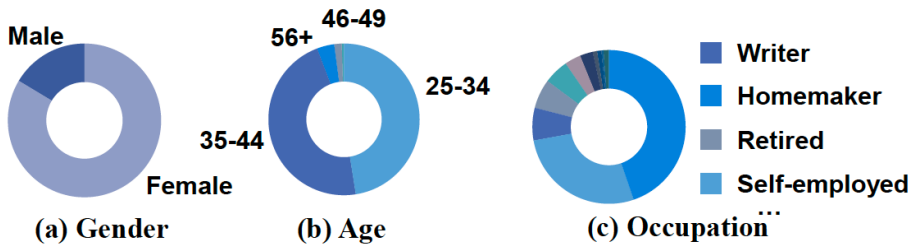
Group	Statistic	Train	Test
User	# Users	939	1,000
User	Avg. profile tokens	247	—
User	Avg. behavior length	32	38
User	Avg. behavior tokens	7,597	9,270
Instruction	# Instructions	6,896	2,174
Instruction	Avg. instruction tokens	46	45
Product	# Products	8,236	—
Product	Avg. product tokens	665	—

Interpretation

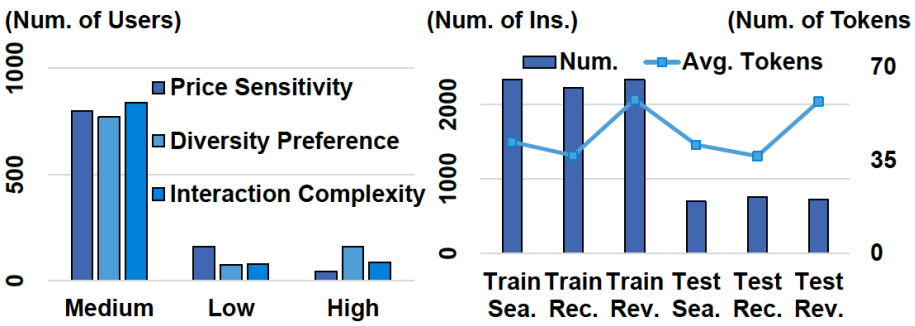
The benchmark is not tiny: each test instance combines a user profile, long behavior history, task instruction, product space, and callable tools.

Benchmark analysis: diversity of users and instructions

Presenter: Eric



(a) Gender (b) Age (c) Occupation



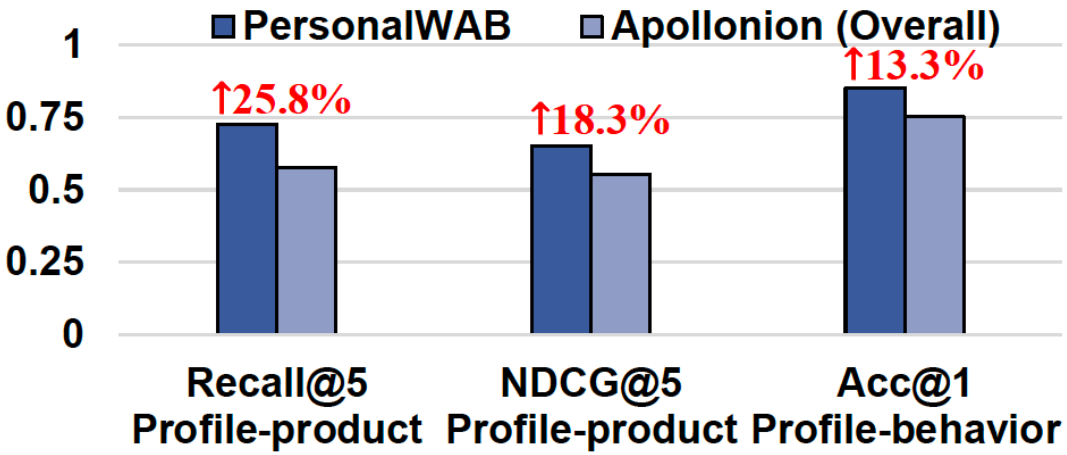
(a) Behaivoral Attributes (b) Instructions

Key observations

- Profiles cover a **wide spread of demographics** and occupations.
- Behavioral attributes include price sensitivity, diversity preference, and interaction complexity.
- Most users fall into the **medium** range, but there are enough **low/high** cases to stress-test personalization.
- Recommendation instructions are usually shorter; review instructions are slightly more expressive and detailed.

Benchmark analysis: are the generated profiles believable?

Presenter: Eric



Profile consistency evaluation

The paper checks whether a generated profile actually matches:

- the user's past behaviors,
- the user's likely interesting products.

Observed gain

PersonalWAB improves over Apollonion by roughly:

- **+25.8%** on Recall@5,
- **+18.3%** on NDCG@5,
- **+13.3%** on Acc@1.

Benchmark strengths and limitations

Presenter: Eric

Why this benchmark matters

- first benchmark centered on **personalized web-agent behavior**
- evaluates both **single-turn** and **multi-turn** interaction
- separates **function choice** from **parameter quality**

What it still abstracts away

- e-commerce only
- function abstraction instead of messy real websites
- simulated profiles and user simulator instead of real deployment

Evaluation takeaway

PersonalWAB is a **clean testbed** for personalization, but not the full web.

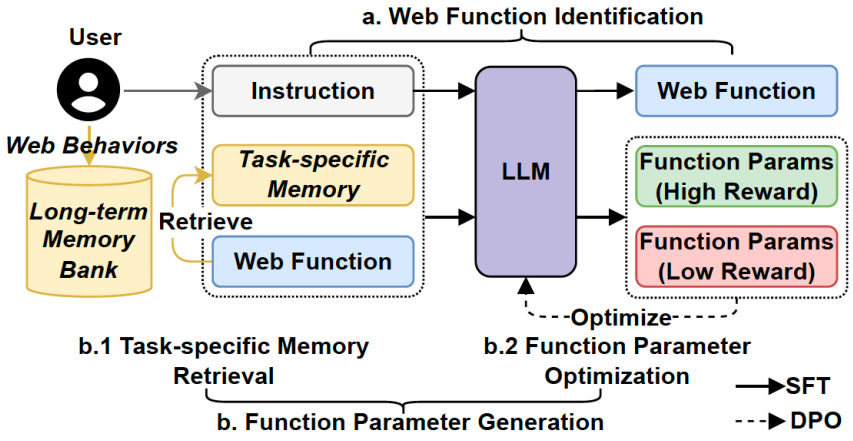
1 Background & Problem Statement

2 Benchmark Construction & Analysis

3 PUMA, Experiments & Conclusion

PUMA: the main method idea

Presenter: Yan



Personalized User Memory-enhanced Alignment

PUMA turns personalization into a two-stage pipeline:

1. identify the right web function,
2. generate the most user-aligned function parameters.

Method intuition

- retrieve the **right slice** of long-term memory,
- then align parameter generation with user preference.

Task-specific memory retrieval

Presenter: Yan

Long-term memory bank

For each user, PUMA stores historical purchases, product metadata, ratings, and reviews.

Retrieval strategy

For a new instruction, it retrieves the **Top-K** most relevant behaviors and then extracts features matched to the chosen function.

Equation (2)

$$M_i = \mathrm{Extract}\big(\mathrm{TopK}(M, \mathrm{sim}(i, m_j), K), f\big)$$

- M: long-term memory bank
- i: current instruction
- f: identified web function

Function parameter optimization: SFT + DPO

Presenter: Yan

Stage 1: supervised fine-tuning

Pseudo-labels are built heuristically:

- search: textual query
- recommendation: recent product IDs from relevant category
- review: actual review text

Preference-pair dataset

$$\mathcal{D}_{\mathrm{DPO}} = \{(p_i^b, p_i^w, x_i)\}_{i=1}^n$$

Best and worst parameter candidates are chosen by result accuracy.

Why Stage 1 matters

SFT gives the model a reasonable initialization before preference-based optimization.

Function parameter optimization: Stage 2 with DPO

Presenter: Yan

Stage 2: DPO

$$\mathcal{L}_{\mathrm{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(p^b \mid x)}{\pi_{\mathrm{ref}}(p^b \mid x)} - \beta \log \frac{\pi_\theta(p^w \mid x)}{\pi_{\mathrm{ref}}(p^w \mid x)}\right)\right]$$

- reward better parameters,
- suppress worse parameters,
- align outputs with user preference.

Intuition

DPO pushes the model toward parameter choices that lead to better personalized outcomes without requiring an explicit reward model at deployment time.

Single-turn results: PUMA leads on overall personalization

Presenter: Yan

Method	Search		Recommendation		Review		Overall	
	F. Acc.	Res Acc	F. Acc.	Res Acc	F. Acc.	Res Acc	F. Acc.	Res Acc
No Memory (gpt-4o)	1.000	0.647	0.092	0.000	1.000	0.444	0.684	0.355
Random Memory (gpt-4o)	0.974	0.640	0.296	0.018	0.996	0.442	0.745	0.357
Last Memory (gpt-4o)	0.937	0.626	0.432	0.028	1.000	0.442	0.782	0.357
Relevant Memory (gpt-4o)	0.928	0.622	0.492	0.030	1.000	0.443	0.800	0.356
ReAct (gpt-4o)	0.903	0.605	0.560	0.027	0.996	0.444	0.815	0.350
RecMind (gpt-4o)	0.981	0.645	0.226	0.017	0.990	0.442	0.721	0.359
PUMA (gpt-4o)	1.000	0.649	0.939	0.048	1.000	0.449	0.979	0.373
PUMA (LLaMA-7B)	**0.996**	**0.652**	**0.987**	**0.054**	**1.000**	**0.538**	**0.994**	**0.406**

Key takeaway

The biggest improvement is in **recommendation function accuracy**, where personalization is most needed and generic tool-calling is not enough.

Multi-turn results: stronger accuracy with controlled interaction

Presenter: Yan

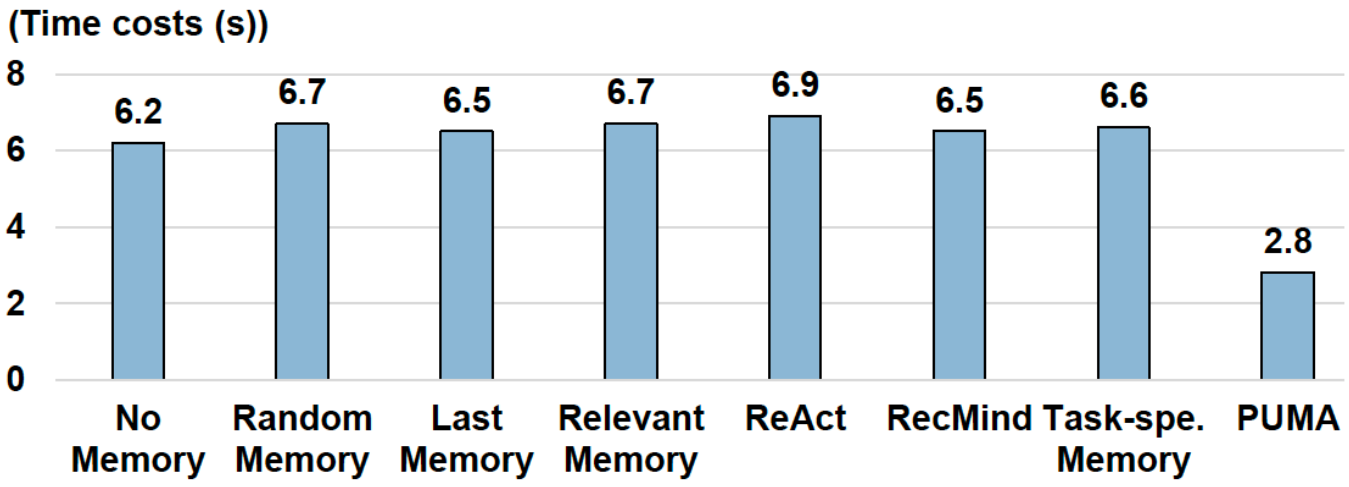
Method	Search			Recommendation			Review			Overall		
	F.	R.	Steps	F.	R.	Steps	F.	R.	Steps	F.	R.	Steps
No Memory	0.996	0.656	2.398	0.096	0.000	2.420	1.000	0.446	2.019	0.685	0.358	2.280
Random Memory	0.999	0.680	4.193	0.703	0.042	4.474	1.000	0.448	2.007	0.896	0.380	3.564
Last Memory	0.996	0.676	4.229	0.708	0.045	4.252	1.000	0.449	2.007	0.897	0.381	3.498
Relevant Memory	0.996	0.686	4.233	0.715	0.042	4.564	0.999	0.448	2.008	0.899	0.383	3.609
ReAct	0.996	0.674	4.657	0.218	0.013	5.468	0.974	0.448	2.129	0.718	0.369	4.098
Reflexion	1.000	0.686	5.406	0.281	0.014	6.145	0.976	0.449	2.145	0.741	0.373	4.579
RecMind	0.997	0.642	6.728	0.347	0.026	6.003	0.997	0.451	2.107	0.771	0.364	4.938
InteRecAgent	0.999	0.642	3.110	0.618	0.022	3.008	1.000	0.447	2.001	0.867	0.362	2.706
PUMA	**0.999**	**0.720**	5.082	**0.984**	**0.052**	3.791	**1.000**	**0.453**	2.002	**0.994**	**0.399**	3.608

Interpretation

In multi-turn settings, memory retrieval helps more, but **PUMA** still achieves the best overall function accuracy and result accuracy.

Efficiency: better personalization does not have to be slower

Presenter: Yan



Reported result

In the single-turn track, GPT-based baselines take about **6.5–6.9 seconds** on average, while **PUMA** takes about **2.8 seconds**.

Why this matters

The method is not just more accurate. It is also more practical for deployment where latency matters.

Conclusion: what should we remember?

Presenter: Yan

- The paper formalizes a new setting: **personalized web agents**.
- **PersonalWAB** provides the first dedicated benchmark for this setting.
- **PUMA** improves performance by combining:
  - task-specific memory retrieval,
  - function identification,
  - and preference-aligned parameter optimization.
- The gains are strongest when the instruction is **ambiguous or underspecified**.

One-sentence summary

If web agents are meant to act *for users*, they must remember who the users are.

Limitations and future work

Presenter: Yan

Limitations

- restricted to e-commerce
- simplified function environment
- profile quality depends on stored behavior history
- privacy and stale-memory issues remain open

Future directions

- broader domains and more realistic web environments
- dynamic preference updating over time
- real user interaction and human-in-the-loop evaluation
- fairness, safety, and privacy-aware personalization

Hongru Cai et al., *Large Language Models Empowered Personalized Web Agents*, WWW 2025.

Thank you!

Questions?